Phase 0: Cấu hình Kotlin + KSP cho dự án — Hướng dẫn từng bước

Mục tiêu Phase 0: dự án Android viết bằng Kotlin (`.kt`), dùng Room với **KSP** để sinh code database.

Điều kiện trước khi làm: dự án tạo từ Android Studio với **AGP 9.2.1**, Gradle **9.4.1**, `minSdk = 33` , `compileSdk = 36` .

Kết quả sau Phase 0: Gradle sync thành công, KSP sinh được code Room, build `assembleDebug` thành công.

Tổng quan các file sẽ chạm

File	Loại	Việc
`gradle/libs.versions.toml`	Sửa	Thêm version `room` , `ksp` , library `room-runtime` , `room-compiler` , plugin `ksp`
`build.gradle.kts` (root)	Sửa	Khai báo plugin `ksp` với `apply false`
`app/build.gradle.kts`	Sửa	Áp dụng plugin `ksp` , khai báo `ksp(libs.room.compiler)`

Step 1) Cơ chế Kotlin trong AGP 9

Giải thích:

- **AGP 9.x** có sẵn **built-in Kotlin** (Kotlin Gradle Plugin **2.2.10** đi kèm). Không cần thêm plugin `org.jetbrains.kotlin.android` .
- File `.kt` đặt trong `app/src/main/java/com/aistudy/aistudy/...` được biên dịch bình thường; thư mục nguồn giữ tên `java` theo mặc định của Android Studio.
- Room cần một bộ xử lý annotation để sinh `AppDatabase_Impl` , `UserDao_Impl` ... từ các class có `@Database` , `@Dao` , `@Entity` . Dự án dùng **KSP** (Kotlin Symbol Processing) cho việc này.
- KSP bản **2.3.x** có version độc lập với version Kotlin, dùng được với built-in Kotlin của AGP 9.

Step 2) Thêm KSP vào `libs.versions.toml`

File: `gradle/libs.versions.toml`

Trong khối `[versions]` , thêm dòng cuối:

```
room = "2.7.2"
ksp = "2.3.12"
```

Trong khối `[libraries]` , thêm dòng cuối:

```
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room" }
```

Trong khối `[plugins]` , thêm dòng cuối:

```
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Khối `[plugins]` sau khi sửa:

```
[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Step 3) Khai báo plugin ở `build.gradle.kts` (root)

Giải thích: File root chỉ khai báo plugin với `apply false` để module `app` áp dụng lại mà không phải ghi version lần nữa.

File: `build.gradle.kts` (thư mục gốc dự án)

Thay toàn bộ nội dung bằng:

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.ksp) apply false
}
```

Step 4) Áp dụng KSP ở `app/build.gradle.kts`

File: `app/build.gradle.kts`

Tìm:

```
plugins {
    alias(libs.plugins.android.application)
}
```

Đổi thành:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.ksp)
}
```

Trong khối `dependencies`, hai dòng Room có dạng:

```
    implementation(libs.room.runtime)
    ksp(libs.room.compiler)
```

Lưu ý:

- `room-runtime` là thư viện chạy lúc runtime; `room-compiler` chỉ chạy lúc build thông qua cấu hình `ksp(...)`.

- `compileOptions` giữ `JavaVersion.VERSION_11`; built-in Kotlin tự lấy `jvmTarget` theo `targetCompatibility`.
- Không cần `room-ktx` vì dự án gọi DAO đồng bộ (`allowMainThreadQueries()` + `Executor`), không dùng coroutine.
- Nội dung đầy đủ của `app/build.gradle.kts` ở trạng thái cuối cùng nằm trong `docs/PHASE_14_STEP_BY_STEP_CODE.md` (Step 1).

Step 5) Quy ước Kotlin dùng trong toàn dự án

Thành phần	Cách viết
Entity Room	`class User { @PrimaryKey(autoGenerate = true) var id: Long = 0; var email: String? = null }`
Index trong `@Entity`	`indices = [Index(value = ["email"], unique = true)]`
Query trả một dòng	`fun findByEmail(email: String): User?`
Query trả danh sách	`fun getAllByUserId(userId: Long): List<QuestionAnswer>`
Lớp tiện ích chỉ chứa hàm/hằng số	`object` (ví dụ `object XpRules`, `object ProfileDefaults`)
Hằng số của Activity/Fragment	`companion object { const val PREFS_AUTH = "auth" }`
Singleton database	`companion object` + `@Volatile private var instance` + `synchronized`
View gán trong `onCreate` / `onViewCreated`	`private lateinit var tvTitle: TextView`
Click listener	`btnSubmit.setOnClickListener { login() }`
Kiểm tra kiểu	`activity is MainActivity`
Lớn nhất / nhỏ nhất	`maxOf(a, b)` / `minOf(a, b)`
Edge-to-edge	`enableEdgeToEdge()`
Mở Activity	`startActivity(Intent(this, MainActivity::class.java))`

Lưu ý về Room + KSP:

- Property Kotlin tên bắt đầu bằng `is` (ví dụ `isCorrect`) sinh setter `setCorrect`, KSP của Room không tìm được setter này và báo `Cannot find setter for property`. Dự án đặt tên property là `correct` và giữ tên cột bằng `@ColumnInfo(name = "isCorrect")` (xem `docs/PHASE_11_STEP_BY_STEP_CODE.md`, entity `QuizItem`).
- Kiểu trả về của query một dòng khai báo nullable (`User?`, `Profile?`) vì Room trả `null` khi không có dòng nào.
- Property entity dùng `var` có giá trị mặc định để Room tạo object bằng constructor rỗng rồi gán từng cột.

Step 6) Build & kiểm tra

1. **File → Sync Project with Gradle Files**.
2. Chạy:

```
.\gradlew.bat :app:assembleDebug
```

3. Kết quả mong đợi: `BUILD SUCCESSFUL`, thư mục `app/build/generated/ksp/debug/kotlin/com/aistudy/aistudy/database/` có `AppDatabase_Impl.kt`, `UserDao_Impl.kt`...

Lỗi thường gặp

Triệu chứng	Nguyên nhân hay gặp	Cách xử lý
`Plugin [id: 'com.google.devtools.ksp'] was not found`	Thiếu `mavenCentral()` / `gradlePluginPortal()` trong `pluginManagement`	Kiểm tra `settings.gradle.kts`
`Unresolved reference: ksp` trong `dependencies`	Chưa áp dụng `alias(libs.plugins.ksp)` ở module `app`	Thêm vào khối `plugins` của `app/build.gradle.kts`
`AppDatabase_Impl does not exist` lúc chạy	Thiếu `ksp(libs.room.compiler)`	Thêm vào `dependencies`, Rebuild
`[ksp] Cannot find setter for property`	Property Room tên `isXxx`	Đổi tên property + `@ColumnInfo(name = "isXxx")`
`[ksp] Cannot find setter for field`	Property entity khai báo `val`	Đổi thành `var` có giá trị mặc định

---

Checklist hoàn thành Phase 0

- [ ] `libs.versions.toml` có `room = "2.7.2"`, `ksp = "2.3.12"`, library `room-runtime`, `room-compiler` và plugin `ksp`
- [ ] Root `build.gradle.kts` có `alias(libs.plugins.ksp) apply false`
- [ ] `app/build.gradle.kts` có `alias(libs.plugins.ksp)` và `ksp(libs.room.compiler)`
- [ ] `assembleDebug` thành công

Tiếp theo: Phase 1 — Main shell (`docs/PHASE_1_STEP_BY_STEP_CODE.md`).